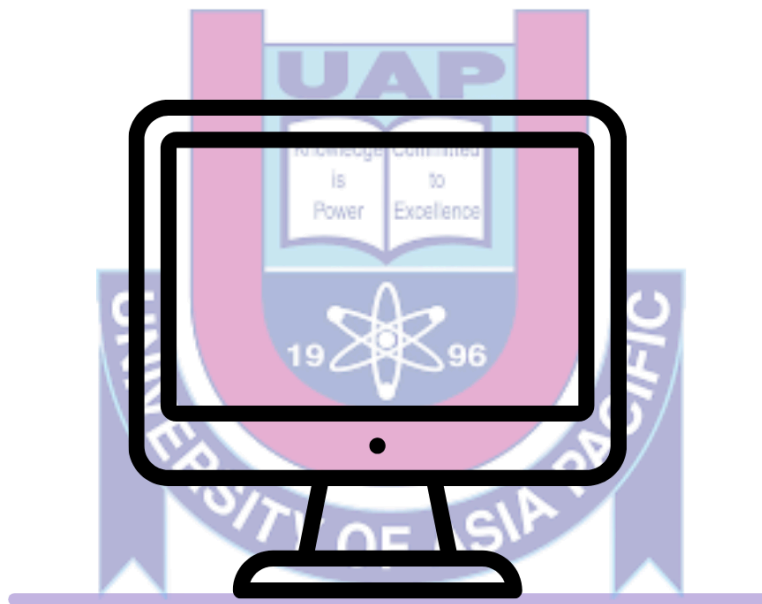


CSE 426



Submitted By:

Mahia Akter Momo
22101122

Sumaya Jahan Othye
22101147

Submitted To:

Md. Rasheduzzaman
Lecturer

**Department of Computer
Science & Engineering
University of Asia Pacific**

Project Name: Catch The Eggs



Version Control Link -> <https://github.com/mahiamOmO/Catch-the-eggs>

Project Developed By:

Team Member 1



Reg ID: 22101122
Sec: C2 4-2 Semester

Team Member 2



Reg ID: 22101147
Sec: C2 4-2 Semester

Table of Content:

Table of Contents.....	2
1. Introduction.....	4
1.1 Problem Statement.....	4
1.2 Project Motivation.....	4
1.3 Background of Computer Graphics.....	4
2. System Design and Methodology.....	5
2.1 System Overview.....	5
2.2 Architecture Diagram.....	5
2.3 Line Drawing Algorithms.....	5
2.3.1 DDA Algorithm (Digital Differential Analyzer).....	5
2.3.2 Bresenham's Line Algorithm.....	6
2.4 Circle and Ellipse Drawing Algorithms.....	6
2.4.1 Midpoint Circle Algorithm.....	6
2.4.2 Midpoint Ellipse Algorithm.....	7
2.5 2D Transformations.....	7
3. Implementation.....	8
3.1 Programming Environment.....	8
3.2 Major Modules and Components.....	8
3.2.1 Initialisation Module.....	8
3.2.2 Game State Machine.....	8
3.2.3 Egg Physics and Collision Detection.....	9
3.2.4 Basket Rendering.....	9
3.2.5 Egg Rendering.....	10
3.2.6 Night Mode.....	10
3.3 Input Handling.....	10
4. Results and Output.....	11
4.1 Game Screens Overview.....	11
4.1.1 Main Menu Screen.....	11
4.1.2 Gameplay Screen.....	11
4.1.3 Pause Screen.....	11
4.1.4 Game Over Screen.....	11

5. Conclusion and Future Scope.....	13
5.1 Conclusion.....	13
5.2 Future Scope.....	13
6. References.....	14
7. Appendices.....	15
Appendix A: GitHub Repository.....	15
Appendix B: Build and Run Instructions.....	15
Appendix C: Controls Reference.....	15
Appendix D: Glossary.....	15

1. Introduction:

1.1 Problem Statement

The objective of this project is to design and implement an interactive 2D arcade game, "Catch-the-Eggs," using the OpenGL graphics library and C++ programming language. The game challenges players to maneuver a basket across the bottom of the screen to catch eggs that fall from the top at varying speeds and positions. The problem addresses the practical application of fundamental computer graphics concepts — including primitive drawing, real-time animation, event-driven input handling, and coordinate transformations — within a fun, interactive context.

1.2 Project Motivation

Learning computer graphics theory in isolation can be abstract and difficult to internalize. Game development provides an ideal medium to bridge theory and practice. By building Catch-the-Eggs, the development team was able to apply line and shape drawing algorithms, 2D transformations, and event-driven rendering in a way that immediately produces visible, engaging results. The project also reinforces understanding of the OpenGL rendering pipeline, callback-based event handling via GLUT/FreeGLUT, and game-loop architecture.

Key motivating goals included:

- Applying Bresenham and DDA algorithms for primitive drawing
- Implementing real-time keyboard and mouse input handling
- Understanding animation through idle-function and timer-based rendering loops
- Practicing state management (menu, play, pause, game-over states)
- Demonstrating night-mode toggling via scene-level color transformations

1.3 Background of Computer Graphics

Computer graphics is the field of study concerned with generating, manipulating, and displaying visual content using computers. It encompasses a wide range of techniques from basic 2D rasterization to complex 3D rendering and shading models.

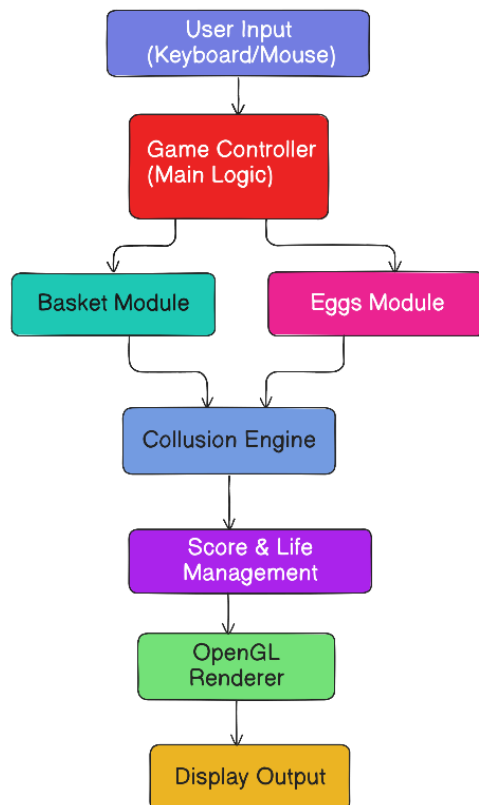
The discipline can broadly be categorised into:

- Raster Graphics: Representing images as grids of pixels (e.g., BMP, PNG). Core algorithms include scan-line fill and rasterization of primitives.
- Vector Graphics: Representing images as mathematical curves and shapes, resolution-independent.
- 2D Graphics: Coordinate geometry, transformations (translation, rotation, scaling), clipping, and viewport mapping.

- 3D Graphics: Projection models (orthographic, perspective), hidden surface removal, lighting and shading (Phong, Gouraud).
- Interactive Graphics: Real-time rendering, event-driven input, and animation — the primary domain of this project.

OpenGL (Open Graphics Library) is a cross-platform, hardware-accelerated graphics API that provides low-level access to GPU-accelerated rendering. Combined with GLUT (GL Utility Toolkit) or its successor FreeGLUT, it enables window creation, event handling, and a complete environment for interactive 2D and 3D applications.

2. System Design and Methodology:



2.1 System Overview:

Component	Responsibility
Main / glutMainLoop	Program entry, window initialisation, callback registration
Display Callback	Renders the current frame based on game state
Timer Callback	Advances egg physics at fixed intervals (game tick)
Mouse Callback	Handles basket movement and menu button clicks
Keyboard Callback	Processes pause (P/ESC) and night-mode (N) keypresses
Game State Machine	Tracks Menu, Play, Pause, GameOver, HighScore, Help states
Score / Timer Module	Manages points, best score persistence, and countdown clock

2.2 Architecture Diagram:

2.3 Line Drawing Algorithms

2.3.1 DDA Algorithm (Digital Differential Analyzer)

The DDA algorithm computes intermediate points of a line by incrementally calculating x and y coordinates. Starting from (x1, y1) to (x2, y2), it determines the number of steps as $\max(|dx|, |dy|)$, then increments x and y by $dx/steps$ and $dy/steps$ per iteration, rounding to the nearest pixel.

DDA Algorithm (Pseudocode):

```
void DDA(int x1, int y1, int x2, int y2) {  
    int dx = x2 - x1, dy = y2 - y1;  
    int steps = max(abs(dx), abs(dy));  
  
    float xInc = dx / (float)steps;  
    float yInc = dy / (float)steps;  
  
    float x = x1, y = y1;
```

```

for (int i = 0; i <= steps; i++) {
    putPixel(round(x), round(y));
    x += xInc;
    y += yInc;
}
}

```

Advantage: Simple to understand and implement. Disadvantage: Uses floating-point arithmetic, making it slower than Bresenham's algorithm on hardware without dedicated floating-point units.

2.3.2 Bresenham's Line Algorithm

Bresenham's algorithm uses only integer arithmetic (addition and subtraction), making it significantly faster than DDA. It maintains a decision parameter (p) that determines whether to increment the minor axis coordinate at each step.

Bresenham's Algorithm (Pseudocode):

```

void bresenham(int x1, int y1, int x2, int y2) {
    int dx = abs(x2 - x1), dy = abs(y2 - y1);
    int p = 2 * dy - dx;
    int x = x1, y = y1;

    for (int i = 0; i <= dx; i++) {
        putPixel(x, y);
        x++;

        if (p < 0)
            p += 2 * dy;
        else {
            y++;

```



```

        p += 2 * (dy - dx);
    }
}
}

```

In Catch-the-Eggs, Bresenham-style logic underlies the drawing of basket edges, HUD borders, and other straight-edged game elements via OpenGL's GL_LINES primitive.

2.4 Circle and Ellipse Drawing Algorithms

2.4.1 Midpoint Circle Algorithm

The midpoint circle algorithm exploits 8-fold symmetry to draw a circle by computing points in only one octant and reflecting them. It uses a decision parameter to choose between candidate pixels at each step.

```

void drawCircle(int xc, int yc, int r) {
    int x = 0, y = r, p = 1 - r;

    while (x <= y) {
        // Plot 8 symmetric points
        putPixel(xc + x, yc + y);
        putPixel(xc - x, yc + y);

        putPixel(xc + x, yc - y);
        putPixel(xc - x, yc - y);

        putPixel(xc + y, yc + x);
        putPixel(xc - y, yc + x);

        putPixel(xc + y, yc - x);
        putPixel(xc - y, yc - x);

        x++;

        if (p < 0)
            p += 2 * x + 1;
        else {
            y--;
            p += 2 * (x - y) + 1;
        }
    }
}

```

}

In the game, eggs are represented as elliptical shapes using OpenGL's `GL_TRIANGLE_FAN` or parametric circle drawing (using `cos/sin` in a loop), which is the practical OpenGL equivalent of the midpoint algorithm.

2.4.2 Midpoint Ellipse Algorithm

For ellipses with semi-axes a (horizontal) and b (vertical), the algorithm is split into two regions based on the gradient of the ellipse boundary. Decision parameters are updated to move pixel-by-pixel along the boundary, again exploiting 4-fold symmetry.

2.5 2D Transformations

The game employs all three fundamental 2D transformations implemented through OpenGL's matrix stack:

Transform	Application in Game	OpenGL Mechanism
Translation	Moving the basket left/right following mouse position; moving eggs downward each timer tick	<code>glTranslatef(tx, ty, 0)</code>
Scaling	Resizing UI elements for different display states (e.g., score text size changes)	<code>glScalef(sx, sy, 1)</code>
Rotation	Rotating egg sprites or decorative elements; potential tilt of basket on fast moves	<code>glRotatef(angle, 0, 0, 1)</code>

These transformations are composed using `glPushMatrix()` / `glPopMatrix()` to isolate transformation contexts for individual game objects, preventing one object's transformation from affecting others.

3. Implementation:

3.1 Programming Environment:

Language	C++ (C++11 or later)
Graphics API	OpenGL 1.1+
Windowing Toolkit	FreeGLUT (Free OpenGL Utility Toolkit)
Compiler	MinGW g++ (Windows) / g++ (Linux/WSL)
IDE	Visual Studio Code / Code::Blocks / any text editor
Build Command	g++ catch_the_eggs.cpp -o game -lfreeglut -lopengl32 -lglu32 -lm
Window Resolution	800 x 600 pixels
Target OS	Windows (primary), Linux/WSL (supported)

3.2 Major Modules and Components

3.2.1 Initialisation Module

The initialisation module sets up the OpenGL context, window, and registers all GLUT callback functions before entering the main event loop.

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);

    glutInitWindowSize(800, 600);
    glutCreateWindow("Catch The Eggs");

    glClearColor(0.1f, 0.1f, 0.3f, 1.0f); // Night sky background
    gluOrtho2D(0, 800, 0, 600);          // 2D orthographic projection

    glutDisplayFunc(display);
    glutIdleFunc(display);

    glutMouseFunc(mouseClick);
    glutPassiveMotionFunc(mouseMove);
    glutKeyboardFunc(keyboard);
}
```

```
glutTimerFunc(16, timerFunc, 0);    // ~60 FPS timer

glutMainLoop();

return 0;
}
```

3.2.2 Game State Machine

The game state is managed via an enumeration and a global state variable. The display callback branches on the current state to determine what to render:

```
enum GameState {
    MENU,
    PLAY,
    PAUSE,
    GAME_OVER,
    HIGH_SCORE,
    HELP
};

GameState currentState = MENU;

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    switch (currentState) {
        case MENU:
            drawMenu();
            break;

        case PLAY:
            drawGame();
            break;
```

```

        case PAUSE:
            drawPause();
            break;

        case GAME_OVER:
            drawGameOver();
            break;

        case HIGH_SCORE:
            drawHighScore();
            break;

        case HELP:
            drawHelp();
            break;
    }

    glutSwapBuffers();
}

```

3.2.3 Egg Physics and Collision Detection

Eggs are stored as structs containing position, speed, and active status. The timer callback updates their positions and checks for collision with the basket or the ground:

```

struct Egg {
    float x, y, speed;
    bool active;
};

Egg eggs[MAX_EGGS];

void timerFunc(int value) {

```

```

for (int i = 0; i < MAX_EGGS; i++) {
    if (!eggs[i].active)
        continue;

    eggs[i].y -= eggs[i].speed; // Translate downward

    // Collision with basket (AABB check)
    if (eggs[i].y < BASKET_TOP &&
        eggs[i].x > basketX - BASKET_W / 2 &&
        eggs[i].x < basketX + BASKET_W / 2) {

        score++;
        eggs[i].active = false;
        spawnEgg(i);
    }

    // Missed - fell below screen
    if (eggs[i].y < 0) {
        eggs[i].active = false;
        spawnEgg(i);
    }
}

glutPostRedisplay();
glutTimerFunc(16, timerFunc, 0);
}

```

3.2.4 Basket Rendering

The basket is drawn as a trapezoid using GL_QUADS, with its horizontal position updated by mouse input. Translation transformation positions the basket at the current mouse x coordinate:

3.2.5 Egg Rendering

```

void drawBasket() {
    glPushMatrix();

```

```

glTranslatef(basketX, BASKET_Y, 0);

glColor3f(0.6f, 0.3f, 0.1f); // Brown basket colour

glBegin(GL_QUADS); // Trapezoid shape
    glVertex2f(-BASKET_W / 2, 0);
    glVertex2f( BASKET_W / 2, 0);
    glVertex2f( BASKET_W / 2 - 10, -BASKET_H);
    glVertex2f(-BASKET_W / 2 + 10, -BASKET_H);
glEnd();

glPopMatrix();
}

```

Each egg is rendered as a filled ellipse using a parametric circle loop — the OpenGL equivalent of the midpoint ellipse algorithm:

```

void drawEgg(float cx, float cy) {
    float rx = 15.0f, ry = 20.0f; // Semi-axes

    glColor3f(1.0f, 0.95f, 0.8f); // Cream colour

    glBegin(GL_TRIANGLE_FAN);

    glVertex2f(cx, cy); // Centre

    for (int i = 0; i <= 360; i++) {
        float angle = i * M_PI / 180.0f;
        glVertex2f(cx + rx * cos(angle),
                    cy + ry * sin(angle));
    }

    glEnd();
}

```

3.2.6 Night Mode

The night mode toggle changes the background clear color and adjusts object colors system-wide by checking a boolean flag during each render call:

```
bool nightMode = false;
```

```
void keyboard(unsigned char key, int x, int y) {  
    if (key == 'n' || key == 'N') {  
        nightMode = !nightMode;  
  
        if (nightMode)  
            glClearColor(0.02f, 0.02f, 0.08f, 1.0f); // Dark night  
        else  
            glClearColor(0.53f, 0.81f, 0.98f, 1.0f); // Day sky  
    }  
}
```

3.3 Input Handling

Input is handled through three GLUT callbacks:

- `glutMouseFunc`: Detects clicks on menu buttons and repositions basket on left-click during gameplay.
- `glutPassiveMotionFunc`: Tracks mouse movement to smoothly follow the cursor with the basket during active play.
- `glutKeyboardFunc`: Handles P / ESC for pause-resume and N for night-mode toggle.
- Arrow keys / A-D: Alternative left-right basket movement registered via `glutSpecialFunc`.

4. Results and Output:

4.1 Game Screens Overview

The game produces an 800x600 pixel interactive window titled "Catch The Eggs". The following describes the visual outputs observed at each game state:

4.1.1 Main Menu Screen

Upon launching, the player is presented with the main menu containing four buttons: START GAME, HIGH SCORE, HELP, and EXIT. The background shows a gradient sky (or dark sky in night mode). Menu buttons are rendered as rounded rectangles with hover effects activated by mouse proximity.

4.1.2 Gameplay Screen

During active play, the screen displays:

- Background scene with sky gradient and ground
- One or more eggs falling from randomised x-positions at the top
- A brown trapezoidal basket following the mouse cursor at the bottom
- HUD overlay: current score (top-left), best score, and countdown timer (top-right)
- Timer ticking down from 1:45 (105 seconds)

4.1.3 Pause Screen

Pressing P or ESC overlays a semi-transparent pause panel showing "PAUSED" with instructions to resume. Egg positions are frozen during pause.

4.1.4 Game Over Screen

When the timer reaches 0:00, the game transitions to the Game Over screen displaying the final score, best score if a new record was set, and options to Play Again or return to the Main Menu.

4.1.5 Night Mode

Toggling night mode with the 'N' key switches the background from a light sky blue to a deep dark blue/black, simulating a night-time environment. All on-screen elements remain visible with adjusted colour values.

4.2 Performance

The game runs smoothly at approximately 60 frames per second on any modern hardware with OpenGL 1.1 support, achieved via the `glutTimerFunc(16, ...)` callback which fires approximately every 16 milliseconds.

Metric	Value
Target Frame Rate	~60 FPS (16ms timer)
Window Resolution	800 x 600 pixels
Game Duration	1 minute 45 seconds (105 seconds)
Egg Speed	Dynamic (increases over time)

Egg Spawn Pattern	Random x-position, fixed spawn y
Basket Width	~80 pixels (trapezoidal)
Collision Detection	AABB (Axis-Aligned Bounding Box)

5. Conclusion and Future Scope:

5.1 Conclusion

The Catch-the-Eggs project successfully demonstrates the practical application of computer graphics fundamentals within a real-time interactive game. Through its development, core concepts including primitive drawing algorithms (DDA, Bresenham, midpoint circle/ellipse), 2D transformations (translation, scaling, rotation), and event-driven input handling were applied and validated in a working software product.

The use of OpenGL and FreeGLUT provided a hardware-accelerated rendering foundation while keeping the API surface manageable for an academic project. The game-loop architecture, state machine design, and modular rendering functions produced a clean and extensible codebase. The project met all stated objectives: interactive gameplay, score tracking, timer-based challenge, best score recording, and a toggle-able night mode.

5.2 Future Scope

Several enhancements could be incorporated in future iterations:

1. **Difficulty Levels:** Introduce Easy, Medium, and Hard modes with varying egg fall speeds and spawn frequencies.
2. **Sound Effects:** Integrate audio feedback for catching eggs, dropping eggs, and game-over using OpenAL or SDL_mixer.
3. **Multiple Egg Types:** Different egg variants with unique point values (golden egg = 5 points, cracked egg = -1 point).
4. **Power-Ups and Obstacles:** Add special items such as a wider-basket power-up, speed-reduction item, or bomb that reduces score.
5. **3D Rendering:** Transition to a 3D perspective view using OpenGL 3.x with perspective projection, textured models, and Phong shading.
6. **Multiplayer Mode:** Side-by-side two-player mode where each player controls a basket using different input keys.
7. **Leaderboard System:** Persistent high score storage using file I/O, retaining the top 10 scores across sessions.

8. Mobile Port: Adapt the game using OpenGL ES for Android/iOS, replacing mouse input with touch event handling.
9. Particle Effects: Add egg-splat animations for missed eggs using point or quad-based particle systems.
10. Shader Support: Upgrade to GLSL shaders for more sophisticated visual effects including glow, shadows, and dynamic lighting.

6. References

11. Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1990). Computer Graphics: Principles and Practice (2nd ed.). Addison-Wesley.
12. Hill, F. S., & Kelley, S. M. (2006). Computer Graphics Using OpenGL (3rd ed.). Pearson Prentice Hall.
13. Hearn, D., & Baker, M. P. (2003). Computer Graphics with OpenGL (3rd ed.). Pearson Education.
14. Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide (8th ed.). Addison-Wesley.
15. Bresenham, J. E. (1965). Algorithm for Computer Control of a Digital Plotter. IBM Systems Journal, 4(1), 25-30.
16. OpenGL Documentation. The Khronos Group. <https://www.khronos.org/opengl/>
17. FreeGLUT Documentation. <http://freeglut.sourceforge.net/>
18. Sutherland, I. E. (1963). Sketchpad: A Man-Machine Graphical Communication System. AFIPS Conference Proceedings.
19. De Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). Computational Geometry: Algorithms and Applications (3rd ed.). Springer.
20. GitHub Repository - Catch-the-Eggs.
<https://github.com/mahiamOmO/Catch-the-eggs>

7. Appendices

Appendix A: GitHub Repository

The complete source code for this project is available at the following GitHub repository:

<https://github.com/mahiamOmO/Catch-the-eggs>

Repository Contents:

- Catch-the-eggs/catch_the_eggs.cpp — Full game source code (C++)
- README.md — Project overview, build instructions, and usage guide
- LICENSE — MIT License

Appendix B: Build and Run Instructions

Prerequisites:

- MinGW g++ compiler (Windows) or g++ (Linux)
- FreeGLUT library installed and linked
- OpenGL 1.1+ compatible GPU and drivers

Build command:

```
g++ catch_the_eggs.cpp -o game.exe -lfreeglut -lopengl32 -lglu32 -lm
```

Run command:

```
game.exe    # Windows ./game    # Linux / WSL
```

Appendix C: Controls Reference

Key / Action	Function
Mouse Move	Smoothly tracks basket to cursor position
Left Mouse Click	Jump basket to clicked x position / Select menu button
Arrow Keys / A, D	Move basket left or right
P or ESC	Pause / Resume game
N	Toggle Night Mode on/off
Start Game (Menu)	Begin a new game session
High Score (Menu)	View best scores achieved
Help (Menu)	Display gameplay instructions
Exit (Menu)	Close the application

Appendix D: Glossary

Term	Definition
OpenGL	Open Graphics Library — a cross-platform graphics rendering API
GLUT / FreeGLUT	OpenGL Utility Toolkit — provides windowing and input callback management
DDA	Digital Differential Analyzer — a line drawing algorithm using incremental floating-point steps
Bresenham	Integer-arithmetic line drawing algorithm devised by Jack Bresenham (1965)
GL_QUADS	OpenGL primitive for drawing four-vertex quadrilateral polygons
GL_TRIANGLE_FAN	OpenGL primitive for drawing filled fan-shaped polygons from a centre vertex
AABB	Axis-Aligned Bounding Box — a simple rectangular collision detection method
Game State Machine	A design pattern managing discrete application states (menu, play, pause, etc.)
glutTimerFunc	GLUT function to schedule a callback after a specified millisecond delay
glPushMatrix	Saves the current OpenGL transformation matrix on a stack
gluOrtho2D	Sets up a 2D orthographic projection for coordinate-based 2D rendering

Sample Input & Output:

```

1 static const int WIN_W = 800;
2 static const int WIN_H = 600;
3 static const float PI = 3.14159265f;
4
5 /* ===== */
6 GAME STATE
7
8 enum GameState { ST_MENU, ST_PLAY, ST_PAUSE, ST_OVER, ST_HIScore, ST_HELP };
9 static GameState gState = ST_MENU;
10 static GameState gLastState = ST_MENU;

```

```

1
2 /* ===== */
3 ITEM TYPES
4
5 enum ItemType {
6     EGG_NORMAL, /* white +1 pt */
7     EGG_BLUE, /* blue +5 pt */
8     EGG_GOLDEN, /* gold +10 pt */
9     POOP, /* brown -10 pt */
10    PWR_BASKET, /* orange bigger basket 10s +3pt bonus */
11    PWR_SLOW, /* cyan slow eggs 8s +3pt bonus */
12    PWR_TIME, /* green +20 seconds +5pt bonus */
13    PWR_SHIELD, /* purple negate 1 poop +2pt bonus */
14 };
15

```

```

1 /* ===== */
2 STRUCTS
3
4 struct Item {
5     float x, y, uy, vx;
6     ItemType type;
7     bool active;
8 };
9
10 struct Chicken {
11     float x, y, speed;
12     int dir;
13     float layTime, layInterval;
14 };
15
16 struct Basket {
17     float cx, y, w;
18 };
19
20 struct Cloud {
21     float x, y, spd;
22 };
23
24 struct Poppy {
25     float x, y, uy, life, maxLife;
26     char text[16];
27     float r, g, b;
28 };
29

```

```

1 /* which melody is active */
2 enum MusicTrack { TRACK_NONE, TRACK_MENU, TRACK_GAME, TRACK_OVER };
3 static MusicTrack gMusicTrack = TRACK_NONE;
4
5 /* current position inside melody */
6 static int gMusicBeatIdx = 0; /* sample index within current beat */
7 static int gMusicBeatSample = 0; /* samples per beat (set when track changes) */
8 static float gMusicCurFreq = 0.f;
9 static int gMusicTotalBeats = 0; /* for one-shot tracks (OVER) */
10 static int gMusicBeatsPlayed = 0;
11
12
13 /* ===== sound-effect queue (lock-free ring, auto writes, callback reads) ===== */
14 struct SfxHole { float freq; int dur; int pos; bool active; };
15 static const int MAX_SFX = 12;
16 static SfxHole gSfx[MAX_SFX] = {};
17 static SDL_SemLock gSfxLock = 0;
18
19 static void sfxTrigger(float freq, float dur_sec, float freq2=0, float dur2=0){
20     SDL_AtomicLock(&gSfxLock);
21     /* find two free slots */
22     int found=0;
23     for(int i=0; i<MAX_SFX; i++){
24         if(gSfx[i].active){
25             if(found==0){
26                 gSfx[i].freq=(int)(dur_sec*AUDIO_SAMPLE_RATE)+0,true;
27                 found++;
28             } else if(freq2>0){
29                 /* second note starts half-way through first */
30                 gSfx[i+1].freq=(int)(dur2*AUDIO_SAMPLE_RATE)+0,true;
31                 found++;
32             } else break;
33         }
34     }
35     SDL_AtomicUnlock(&gSfxLock);
36 }
37

```

```

1 /* wind */
2 static bool gWindOn = false;
3 static float gWindT = 0.f;
4 static float gWindF = 0.f;
5
6 /* night mode */
7 static bool gNight = false;
8
9 /* animation clock */
10 static float gAnia = 0.f;
11
12 static const float BW_NORMAL = 110.f;
13 static const float BW_BIG = 210.f;
14 static const float BH = 44.f;
15 static const float BSPEED = 380.f;
16
17 static bool gKeys[256] = {};
18 static bool gSpec[256] = {};
19 static int gHover = -1;
20 static float gLastTime = 0.f;
21
22 static float gStarX[120], gStarY[120], gStarBr[120];
23

```

```

1 /* ===== */
2 AUDIO - SDL2 procedural synthesis (Linux + Windows)
3 ===== */
4
5 static const int AUDIO_SAMPLE_RATE = 44100;
6 static const int AUDIO_CHANNELS = 1; /* mono */
7 static SDL_AudioDeviceID gAudioDev = 0;
8
9 /* ===== music state (written only from main thread, read in callback) ===== */
10 static bool gMusicOn = true;
11 static bool gMusicPlaying = false;
12
13 /* Game BGM: cheerful major-key 16-note loop */
14 static const float GAME_MELODY[] = {
15     523,659,784,880, 784,659,784,523,
16     392,523,659,784, 659,523,659,392
17 };
18 static const int N_GAME_NOTES = 16;
19 static const float GAME_BEAT_DUR = 0.22f; /* seconds per note */
20

```

```

1 /* ===== */
2 GLOBALS
3
4 static const int N_STICKS = 2;
5 static const float STICK_W[N_STICKS] = { WIN_W - 90.f, WIN_W - 230.f };
6
7 static std::vector<Chicken> gChens;
8 static Basket gBasket;
9 static std::vector<Item> gItems;
10 static std::vector<Cloud> gClouds;
11 static std::vector<Poppy> gPoppys;
12
13 static int gScore = 0;
14 static int gHiScore = 0;
15 static float gTime = 0.f; /* t since */
16
17 /* combo */
18 static int gCombo = 0;
19 static float gComboTimer = 0.f;
20
21 /* power timers */
22 static bool gSlowOn = false; static float gSlowT = 0.f;
23 static bool gTimeOn = false; static float gTimeT = 0.f;
24 static bool gShield = false;
25

```

```

1 /* ===== public music API (called from game code) ===== */
2 static void musicPlay() { gMusicPlaying=true; }
3 static void musicStop() { gMusicPlaying=false; }
4 static void musicToggle() {
5     gMusicOn=!gMusicOn;
6     if(!gMusicOn) musicStop();
7     else musicPlay();
8 }
9 static void playMenuMusic() { setMusicTrack(TRACK_MENU); }
10 static void playGameOverMusic() { setMusicTrack(TRACK_OVER); }
11 static void playGameMusic() { /* game track starts at initGame() */
12
13 /* ===== sound effects ===== */
14 static void soundCatch(ItemType t){
15     switch(t){
16     case EGG_NORMAL: sfxTrigger(880, 0.06f); break;
17     case EGG_BLUE: sfxTrigger(1100, 0.08f); break;
18     case EGG_GOLDEN: sfxTrigger(1400, 0.12f, 1760, 0.08f); break;
19     case POOP: sfxTrigger(180, 0.18f); break;
20     default: sfxTrigger(660, 0.07f, 990, 0.07f); break;
21     }
22 }
23

```

```

1 /* ===== */
2 TREE
3
4 static void drawTree(float cx, float groundY, float scale=1.f){
5     float tw=12*scale, th=28*scale;
6     quad(cx-tw/2, groundY, tw, th, 0.42f, 0.26f, 0.85f);
7     glColor3f(0.12f, 0.68f, 0.12f);
8     glBegin(GL_TRIANGLES);
9     glVertex2f(cx+38*scale, groundY+th);
10    glVertex2f(cx+38*scale, groundY+th);
11    glVertex2f(cx, groundY+th+46*scale);
12    glEnd();
13    glColor3f(0.10f, 0.58f, 0.10f);
14    glBegin(GL_TRIANGLES);
15    glVertex2f(cx+27*scale, groundY+th+27*scale);
16    glVertex2f(cx+27*scale, groundY+th+27*scale);
17    glVertex2f(cx, groundY+th+66*scale);
18    glEnd();
19    glColor3f(0.15f, 0.68f, 0.15f);
20    glBegin(GL_TRIANGLES);
21    glVertex2f(cx+17*scale, groundY+th+58*scale);
22    glVertex2f(cx+17*scale, groundY+th+58*scale);
23    glVertex2f(cx, groundY+th+82*scale);
24    glEnd();
25 }
26

```

```

1 /* ===== */
2 PAUSE
3 ===== */
4
5 static void drawPause(){
6     darkOverlay(0.65f);
7     quad(WIN_W/2-130, WIN_H/2-140, 260, 240, 0.06f, 0.06f, 0.20f);
8     quadLine(WIN_W/2-130, WIN_H/2-140, 260, 240, 0.9f, 0.75f, 0.1f, 2);
9     txtBig(WIN_W/2-52, WIN_H/2+82, "PAUSED", 1.f, 1.f, 0.f);
10    float bx=WIN_W/2-80, bw=160, bh=42;
11    button(bx, WIN_H/2+30, bw, bh, "RESUME", gHover==0);
12    button(bx, WIN_H/2-20, bw, bh, "RESTART", gHover==1);
13    button(bx, WIN_H/2-70, bw, bh, "MENU", gHover==2);
14    button(bx, WIN_H/2-120, bw, bh, "EXIT", gHover==3);
15 }

```

```

1 /* ===== */
2 HOUSE
3 ===== */
4
5 static void drawHouse(float x, float groundY, float scale=1.f){
6     float w=90*scale, h=70*scale, rx=x-w/2;
7     /* walls */
8     glBegin(GL_QUADS);
9     glColor3f(0.88f, 0.76f, 0.60f); glVertex2f(rx, groundY);
10    glColor3f(0.88f, 0.76f, 0.60f); glVertex2f(rx+w, groundY);
11    glColor3f(0.78f, 0.65f, 0.50f); glVertex2f(rx+w, groundY+h);
12    glColor3f(0.78f, 0.65f, 0.50f); glVertex2f(rx, groundY+h);
13    glEnd();
14    /* roof */
15    glColor3f(0.72f, 0.20f, 0.14f);
16    glBegin(GL_TRIANGLES);
17    glVertex2f(rx+w*8*scale, groundY+h);
18    glVertex2f(x, groundY+h+52*scale);
19    glEnd();
20    glColor3f(0.55f, 0.14f, 0.09f);
21    glBegin(GL_TRIANGLES);
22    glVertex2f(x-6*scale, groundY+h+46*scale);
23    glVertex2f(x+6*scale, groundY+h+46*scale);
24    glVertex2f(x, groundY+h+52*scale);
25    glEnd();

```

```

1 /* ===== */
2 PAUSE
3 ===== */
4
5 static void drawPause(){
6     darkOverlay(0.65f);
7     quad(WIN_W/2-130, WIN_H/2-140, 260, 240, 0.06f, 0.06f, 0.20f);
8     quadLine(WIN_W/2-130, WIN_H/2-140, 260, 240, 0.9f, 0.75f, 0.1f, 2);
9     txtBig(WIN_W/2-52, WIN_H/2+82, "PAUSED", 1.f, 1.f, 0.f);
10    float bx=WIN_W/2-80, bw=160, bh=42;
11    button(bx, WIN_H/2+30, bw, bh, "RESUME", gHover==0);
12    button(bx, WIN_H/2-20, bw, bh, "RESTART", gHover==1);
13    button(bx, WIN_H/2-70, bw, bh, "MENU", gHover==2);
14    button(bx, WIN_H/2-120, bw, bh, "EXIT", gHover==3);
15 }

```

